

Chapter 2 Computer Performance

2.1 Understanding performance

2.2 Calculation of performance

2.3 Amdahl's Law

2.4 Standard Procedures for Evaluating Performance

2.5 Standard organization for performance evaluation

Course information in the cloud drive

- ✖ <https://www.dropbox.com/scl/fo/9o8x6abqwk46ngalgkdq3/AFBWm70ABZqVbIFmMeCAQtg?rlkey=jfhmlgs1rkmtogvhexks5x9tc&st=npj61kmc&dl=0>
- ✖ Course , HW and Score status
- ✖ Student check Line Group

2.1 Understand performance

- ✖ The performance of a single computer
- ✖ Relative performance between different computers

2.1.1 The performance of a single computer

✖ Quantification and standards

Vehicle type	Passenger capacity	Extremely fast (km/hr)	0-100km/h hasten (Sec)
Formula 1 car	1	375	3
sports car	2	300	4.2
Sedan	4	250	9.1
SUV	9	180	13.1

Performance comparison of various cars

2.1.1 The performance of a single computer

✖ Quantification and standards

Vehicle type	Time required for one way(hr)	Average number of passengers per hour
Formula 1 car	0.24	4.17
sports car	0.3	6.67
Sedan	0.36	11.11
SUV	0.5	18

The time and passenger efficiency required for various cars to travel between the two places are 90 kilometers apart

2.1.1 The performance of a single computer

- ✖ Quantification and standards
- ✖ Using data to express performance is a process of quantification.
- ✖ When the calculation method is the same and the units are the same, we can easily understand the comparison results from the numbers.
- ✖ Even if there are no more than two vehicles compared, we can understand the characteristics of each vehicle.
- ✖ The method of generating numbers is a standard.

2.1.1 The performance of a single computer

- ✖ Two angles - execution time and generation volume
- ✖ Execution time
- ✖ The time it takes for a job to be completed from the start of execution.
- ✖ Throughput
- ✖ The maximum number of times a particular task can be completed per unit of time.

2.1 Understand performance

- ✖ The performance of a single computer
- ✖ Relative performance between different computers

2.1.2 Relative performance between different computers

- ✖ There are many times when you think, "Which computer is faster? How fast?" This kind of vague conclusion is more useful.
- ✖ To compare the performance of two different systems, the measurement criteria must also be considered, and a more unbiased approach is to run the same program on two different systems.

2.1.2 Relative performance between different computers

- ✖ Suppose the same bubble sort program is executed on two different systems, A and B:
- ✖ The data to be sorted is n records.
- ✖ It takes x seconds for system A to complete the sorting of n data.
- ✖ It takes y seconds for the B system to complete the sorting of n data.
- ✖ The relationship between runtime and performance is:

$$Performance = \frac{1}{Execution\ Time}$$

2.1.2 Relative performance between different computers

- ✖ The performance of System A is:

$$Performance_A = \frac{1}{x}$$

- ✖ The performance of system B is:

$$Performance_B = \frac{1}{y}$$

2.1.2 Relative performance between different computers

✖ If System A performs better than System B, then:

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\frac{1}{x}}{\frac{1}{y}} = \frac{y}{x} = z$$

The performance of system A
is z times that of system B

2.1.2 Relative performance between different computers

✖ If System A is an improved version of System B, then:

$$\frac{Performance_A - Performance_B}{Performance_B} = \frac{\frac{1}{x} - \frac{1}{y}}{\frac{1}{y}} = \left(\frac{y}{x} - 1 \right) \times 100\% = w\%$$

System A performance is $w\%$
faster than System B

2.2 Performance calculations

- ✖ **Basic performance calculations**
- ✖ The performance criterion is based on the number of instructions executed per unit of time
- ✖ How average performance is calculated
- ✖ Evaluation of relative performance

2.2.1 Basic performance calculations

- ✖ Basic Performance Formula
- ✖ Execution time can also be called CPU execution time for a program, abbreviated as CPU time.
- ✖ CPU time calculation considerations for the program:
 - ✖ Clock cycle time
 - ✖ Clock cycles per instruction (CPI)
 - ✖ Number of instructions in the program (IC)

2.2.1 Basic performance calculations

- ✖ Basic Performance Formula
- ✖ Clock cycle: In computer hardware, CPU work must be triggered or started by a regular signal, the frequency of these signals is called clock rate, the higher the clock frequency, the faster the speed; The reciprocal is the clock period. The CPU can perform a very basic operation in each clock cycle ◦

2.2.1 Basic performance calculations

- ✖ Basic Performance Formula
- ✖ **Clock cycles of a single instruction:** The command of a program is not necessarily the basic action in the CPU circuit, in fact, it usually takes several smaller execution cycles to complete an instruction; These execution cycles are driven by the clock signal provided by the hardware, so the length of the command can be expressed in clock cycles.

2.2.1 Basic performance calculations

- ✖ Basic Performance Formula
- ✖ Different types of instructions may require different clock cycles, so we use their average value to represent the number of clock cycles of each instruction in the program, which is called the average number of clock cycles of a single instruction.
- ✖ Different programs have different instruction compositions, so there may be different CPIs.

2.2.1 Basic performance calculations

✖ Basic Performance Formula

- ✖ Number of Commands in a Program: From a program's point of view, an instruction is the most basic unit, because in the program structure, whether it is a main program, a routine, or a macro, it is composed of a series of instructions. The number of times the program command is executed will inevitably directly affect the length of the program execution time.

2.2.1 Basic performance calculations

- ✖ Basic Performance Formula

- ✖ Formula for CPU Time:

- ✖ Expressed in clock period:

$$CPU\ time = clock\ cycle\ time \times CPI \times IC$$

- ✖ Expressed in clock frequency:

$$CPU\ time = \frac{CPI \times IC}{clock\ rate}$$

2.2.1 Basic performance calculations

- ✖ Analysis of factors affecting performance
- ✖ The clock period (frequency), the average clock period (CPI) of a single instruction, and the number of program instructions (IC) together determine the performance of the CPU.
- ✖ These three may affect each other, and changing any one may affect the other two.
- ✖ It's almost impossible to improve CPU performance from a single approach.

2.2.1 Basic performance calculations

✖ Analysis of factors affecting performance

$$CPU \text{ execution time} = \frac{\text{Seconds}}{\text{Program}} = \overset{\text{Item 1}}{\frac{\text{Seconds}}{\text{Cycle}}} \times \overset{\text{Item 2}}{\frac{\text{Cycles}}{\text{Instruction}}} \times \overset{\text{Item 3}}{\frac{\text{Instructions}}{\text{Program}}}$$

Second : time

Program : The program executed by the CPU

Instruction : The command of the program being executed (count)

Cycle : Time Clock (count)

2.2.1 Basic performance calculations

- ✖ Analysis of factors affecting performance
- ✖ The first item: the operating clock frequency or the period of a single clock.
- ✖ Item 2: Average clock period of a single instruction.
- ✖ Item 3: The number of commands in a program.

2.2.1 Basic performance calculations

- ✖ The first item - the operating clock frequency or the period of a single clock:
- ✖ On the surface, complex hardware circuits can provide many high-performance instructions for programs to apply, which should be of great help to overall performance.
- ✖ But in reality, the complexity of a chip has a lot to do with the frequency it can withstand.
- ✖ The more complex the circuit, the lower the clock frequency that can be applied to it, the longer the clock period will be. On the other hand, a simple circuit may not provide powerful instructions, but it can significantly increase the clock frequency and shorten the time of each clock cycle.

2.2.1 Basic performance calculations

- ✖ The second item - the average clock period of a single instruction is the second factor affecting CPU time, which is related to:
- ✖ Instruction set:
Different ISAs affect the average clock period of the instruction.
- ✖ Single-cycle
- ✖ Multi-cycle
- ✖
- ✖ Hardware Structure:
- ✖ Each machine code instruction corresponds to the circuit that implements the instruction.
- ✖ At the same clock frequency, a simple circuit requires much more clock cycles than a complex hardware circuit that uses a large number of logic gates.

2.2.1 Basic performance calculations

- ✖ The third item - the number of instructions in a program is the first factor affecting CPU time, related to:
- ✖ Programming Style :
 - ✖ The way the program is written has the greatest impact on the number of program instructions (ICs), especially if different algorithms are chosen to write programs, the difference caused by the difference is even greater than that of different orders of magnitude ◦
- + compiler :
 - ✖ Convert high-level language programs to machine language programs. The number of instructions is reduced by optimization by the compiler, which affects the eventual execution performance of the program.

2.2.1 Basic performance calculations

+ Instruction set :

- × Computer processors are generally designed according to the Instruction Set Architecture (ISA) of their choice, and whether the CPU designed according to this has the right hardware unit to directly meet the calculations required by the program will affect the number of instructions required to complete the task.

2.2.1 Basic performance calculations

✖ Analysis of factors affecting performance

Hardware or software section	Impact on performance
algorithm	Affects the number of commands that the program must execute and how much I/O is generated
programming languages, compilers, and algorithms Computer instruction set structure	Decide how many machine instructions will be generated
Processor and memory system	Determine the time it takes to execute each command, as well as the clock frequency
I/O System (Hardware and Operating System)	Determine the speed at which each I/O is processed

Factors that affect program execution performance

2.2 Performance calculations

- ✖ Basic performance calculations
- ✖ The performance criterion is based on the number of instructions executed per unit of time
- ✖ How average performance is calculated
- ✖ Evaluation of relative performance

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

- ✖ Millions of instructions per second (MIPS)
 - + *MIPS* (Million Instructions Per Second) , Defined as "how many million commands a computer can execute per second".

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

✖ Millions of instructions per second (MIPS)

$$\begin{aligned} MIPS &= \frac{IC}{CPU\ time \times 10^6} \\ &= \frac{IC}{IC \times CPI \times clock\ cycle\ time \times 10^6} \\ &= \frac{1}{CPI \times clock\ cycle\ time \times 10^6} \\ &= \frac{clock\ rate}{CPI \times 10^6} \end{aligned}$$

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

- ✖ Millions of instructions per second (MIPS)
 - + Only CPI and clock frequency are required to calculate MIPS (Or clock cycles) However, the number of instructions IC and instruction set of the program are not considered.
 - + If MIPS is used as the only criterion for evaluating the performance of a computer system and is used to evaluate the execution of a single program, it is possible to get incorrect results.

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

【example 1】

Suppose there is a computer A with a clock period of 2ns, but there are three classes of instructions with different CPIs, as shown in the following table:

Command type	CPI
Category A	1
Category B	2
Category C	3

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

【Example 1】

Now, in order to run a program, the source program is compiled with two different compilers (compiler 1 and compiler 2), assuming that the compiled program contains the following number of various instructions:

compiled program	The number of commands of each type		
	A	B	C
Program 1 (use compiler 1)	5,000	1,000	1,000
Program 2 (use compiler 2)	10,000	1,000	1,000

Which compiler generates programs on the computer with better performance?

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

【Answer】

Comparing CPU Times:

The number of CPU clock cycles of the program:

$$CPU\ clock\ cycles_1 = (5,000 \times 1 + 1,000 \times 2 + 1,000 \times 3) = 10,000$$

$$CPU\ clock\ cycles_2 = (10,000 \times 1 + 1,000 \times 2 + 1,000 \times 3) = 15,000$$

The CPU time of the program :

$$CPU\ time_1 = 10,000 \times 2ns = 20\mu s$$

$$CPU\ time_2 = 15,000 \times 2ns = 30\mu s$$

Since $CPU\ time_1 = 20\mu s < 30\mu s = CPU\ time_2$,

The first compiler runs faster!

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

【Answer】

Comparing the MIPS of the two:

$$MIPS_1 = \frac{(5,000 + 1,000 + 1,000)}{20\mu \times 10^6} = 350$$

$$MIPS_2 = \frac{(10,000 + 1,000 + 1,000)}{30\mu \times 10^6} = 400$$

The program compiled by the first compiler "looks" to be less performant!

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

- ✖ Millions of floating-point operations per second (MFLOPS)
 - + *MFLOPS* (Million Floating-point Operations Per Second) : Used to evaluate programs with floating-point operations.
 - ✖ Singlex accuracy (single precision)
 - ✖ Double the precision (double precision)

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

- ✖ Millions of floating-point operations per second (MFLOPS)

$$\text{MFLOPS} = \frac{OP}{CPU\ time \times 10^6}$$

OP : The number of floating-point operations in the program

2.2.2 The performance criterion is based on the number of instructions executed per unit of time

- ✖ Millions of floating-point operations per second (MFLOPS)
 - + *MFLOPS* It also has the same problems as MIPS.
 - + Only CPI and clock frequency are required, and the number of instructions in the program IC and instruction set are not considered.
 - + Using MFLOPS to evaluate a program with little to no floating-point arithmetic may result in zero results.
 - + MIPS and MFLOPS are only a useful standard unit if they are of the same standard.

2.2 Performance calculations

- ✖ Basic performance calculations
- ✖ The performance criterion is based on the number of instructions executed per unit of time
- ✖ How average performance is calculated
- ✖ Evaluation of relative performance

2.2.3 How average performance is calculated

- ✖ From the concept of "mean" in statistics ◦
- ✖ Its meaning is "to simplify all values (CPU time or MIPS) of a group of data into a single value or represent the average level of a group of data, so that the results can be compared with each other, making the display of results more meaningful and in line with the needs of the evaluator."

2.2.3 How average performance is calculated

✖ Arithmetic mean (AM)

$$AM = \frac{1}{n} \sum_{i=1}^n Time_i$$

n : The number of programs that the computer system will run

$Time_i$: CPU time of the i th program

2.2.3 How average performance is calculated

✖ Weighted arithmetic mean (*WAM*)

+ Consider the ratio of the number of executions of each program to the total number of times (or frequency of program execution)

+ Weighting's concept

$$WAM = \sum_{i=1}^n Weight_i \times Time_i$$

n : The total number of programs included in the assessment

$Time_i$: CPU time of the i th program

$Weight_i$: corresponding weights

2.2.3 How average performance is calculated

✖ Harmonic Mean(HM)

- + Performance is expressed in terms of rate ◦
- + It is a reciprocal relationship with AM ◦

$$HM = \frac{n}{\sum_{i=1}^n \frac{1}{Rate_i}} \qquad HM = \frac{1}{AM}$$

n : The number of programs that are executed

$Rate_i$: The rate of the i th program (It is a reciprocal relationship with CPU time)

2.2.3 How average performance is calculated

- ✖ weighted harmonic mean(*WHM*)
 - + Weighting concept

$$WHM = \frac{1}{\sum_{i=1}^n \frac{Weight_i}{Rate_i}}$$

n : The number of programs that are executed

$Rate_i$: The rate of the i th program (CPU time)

$Weight_i$: corresponding weights

2.2 Performance calculations

- ✖ Basic performance calculations
- ✖ The performance criterion is based on the number of instructions executed per unit of time
- ✖ How average performance is calculated
- ✖ Evaluation of relative performance

2.2.4 Evaluation of relative performance

✖ Relative performance and normalization

	A computer CPU time	B computer CPU time	According to computer A Normalize multiples		According to computer B Normalize multiples	
			A	B	A	B
Program x	10	200	1	20	0.05	1
Program y	5000	250	1	0.05	20	1
AM	2505	225	1	10.025	10.025	1

CPU time and AM of x and y programs running on different computers

2.2.4 Evaluation of relative performance

✖ Geometric Mean(GM)

- + The arithmetic average of multiples is susceptible to a single program with a higher multiple, resulting in a situation where the actual result deviates significantly ◦
- + This problem can be solved by using calculated geometric means ◦

$$GM = \sqrt[n]{\prod_{i=1}^n Time_i}$$

n : Total number of data

$Time_i$: CPU time for item i , after normalization

2.2.4 Evaluation of relative performance

✖ Geometric Mean(GM)

	A computer's CPU time	CPU time of computer B	According to computer A Normalize multiples		According to computer B Normalize multiples	
			A	B	A	B
Program x	10	200	1	20	0.05	1
Program y	5000	250	1	0.05	20	1
GM	223.6	223.6	1	1	1	1

The CPU time and GM of the x and y programs running on different computers

2.2.4 Evaluation of relative performance

✖ Summary of various performance calculation methods

title	formula
CPU time	$CPU\ time = \frac{IC \times CPI}{Clock\ rate}$
MIPS	$MIPS = \frac{IC}{CPU\ time \times 10^6}$
MFLOPS	$MFLOPS = \frac{OP}{CPU\ time \times 10^6}$

2.2.4 Evaluation of relative performance

✖ Summary of various performance calculation methods

Mean name	Abbreviation	formula
arithmetic mean	AM	$AM = \frac{1}{n} \sum_{i=1}^n Time_i$
Weighted arithmetic mean	WAM	$WAM = \sum_{i=1}^n Weight_i \times Time_i$
**Harmonic Mean	HM	$HM = \frac{n}{\sum_{i=1}^n \frac{1}{Rate_i}}$

2.2.4 Evaluation of relative performance

✖ Summary of various performance calculation methods

Mean name	Abbreviation	formula
Weights harmonize the mean	WHM	$WHM = \frac{n}{\sum_{i=1}^n \frac{Weight_i}{Rate_i}}$
Geometric mean	GM	$GM = \sqrt[n]{\prod_{i=1}^n Time_i}$

2.3 Amdahl's Law

- ✖ When a local system is improved in a faster way to improve overall system performance, the extent to which it can be improved is limited by the ratio of the "**time it takes to actually run the local system**" compared to the "**overall system execution time**" before the improvement.
- ✖ example :
 - + If addition operations only take up 30% of the execution time in the original program, then no matter how hard you try to improve the performance of addition, you can only improve the overall system performance by up to 30%.

2.3 Amdahl's Law

✖ Definition of Amdahl's theorem :

$$S = \frac{1}{(1 - f) + \frac{f}{k}}$$

- + S : It represents the growth rate of the overall system
- + F : It is the proportion of system improvement in all work
- + K :Indicates the performance of the improved part compared to the growth rate multiple before the improvement.

2.3 Amdahl's Law

【Example 2】

A system manufacturer wanted to launch a new peripheral, and their idea was to improve some of the past models before **shipping**, but due to development funding **constraints**, they had to make trade-offs for the improvements:

1. Only improve CPU time

2. Only improve I/O time

Assuming that the above improvements are 5 times, and the original peripherals and computers take 50ms CPU time and 150ms I/O time to do data transmission, if you are their decision-maker, can you give advice to the manufacturer?

2.3 Amdahl's Law

【Answer】

1. If only CPU time is improved

According to the definition of Amdahl's theorem, "improving CPU time" is a part of system improvement, so we can first find the proportion of CPU time (i.e., the upper limit of the overall performance that can be improved after improving the CPU) :

$$f_{CPU} = \frac{50ms}{50ms + 150ms} = \frac{50}{200} = \frac{1}{4}$$

After improving CPU time, its performance is a multiple compared to the original :

$$Speed\ up = \frac{1}{(1 - f_{CPU}) + \frac{f_{CPU}}{k}} = \frac{1}{\left(1 - \frac{1}{4}\right) + \frac{\frac{1}{4}}{5}} = 1.25$$

2.3 Amdahl's Law

【Answer】

2. Only improve I/O time

Similarly, first find the proportion of I/O time

$$f_{I/O} = \frac{150ms}{50ms + 150ms} = \frac{150}{200} = \frac{3}{4}$$

The improved I/O time results in performance by a factor of 1 compared to the original

$$Speed\ up = \frac{1}{(1 - f_{I/O}) + \frac{f_{I/O}}{k}} = \frac{1}{(1 - \frac{3}{4}) + \frac{\frac{3}{4}}{5}} = 2.5$$

Conclusion: It is recommended to improve I/O time because the benefits obtained are greater.

2.3 Amdahl's Law

✖ How much can the system be improved?

In an ideal world, assuming an "infinite" improvement, then :

$$S = \frac{1}{(1-f) + \frac{f}{\infty}} = \frac{1}{1-f}$$

+ The largest system performance improvement multiple is the reciprocal of the percentage of unimproved parts.

2.4 Standard procedure for evaluating performance

- ✖ Understand the performance evaluation process
(Benchmark)
- ✖ Three famous evaluation programs
- ✖ Problems with the evaluation process

2.4.1 Understand the performance evaluation process (Benchmark)

- ✖ What kind of program is considered a standard program for evaluating computer performance?
- ✖ A single application often uses a large number of specific types of computing and delivers performance that is only a fraction of the overall system.
- ✖ These applications are likely to vary from one operating system or compiler to another on computers with different architectures, and do not represent "the same program".
- ✖ Portability issues.

2.4.1 Understand the performance evaluation process (Benchmark)

- ✖ Evaluating computer performance requires a hybrid program that can cover a wide range of calculations (mixture programs).
 - + The code should be short and simple.
 - + Can be performed on different platforms.
 - + Desktop computers, laptop laptops, portable embedded system devices.
- Such a hybrid program is called a synthetic performance evaluation program (synthetic benchmarks) or the evaluation program (benchmark suite) ,The single program that makes up the hybrid program is only called the evaluation program (benchmark).

2.4 Standard procedure for evaluating performance

- ✖ Understand the performance evaluation process
(Benchmark)
- ✖ Three famous evaluation programs
- ✖ Problems with the evaluation process

2.4.2 Three famous evaluation programs

✖ Whetstone

- + Whetstone was first published in 1976 by Harold J. Curnow and Brian A. Wichman at the National Physical Laboratory in the United Kingdom. It is written in the Algol 60 programming language and emphasizes intensive floating-point operations, so it makes a lot of calls to trigonometric and exponential functions in the library.
- + The results measured by Whetstone are called **WIPS** (Whetstone instructions per second) , You can also use "1,000 Whetstone instructions per second" according to the order of speed of the computer. (Kilo-Whetstone instructions per second , KWIPS) or "Millions of Whetstone Instructions per Second" (MWIPS) is expressed in units.

2.4.2 Three famous evaluation programs

✖ Linpack

- + Linpack (linear algebra package) It was proposed by Jim Pool, director of the Institute of Mathematics at Argonne National Laboratory in the United States, in a series of informal discussions, and was established in 1974 after evaluation, which is a set of mathematical software packages that use double-precision floating-point operations to solve mathematical equations in linear algebraic systems. The lab later proposed the Linpack project, which was approved and funded by the National Science Foundation, and the Argonne Department of Energy provided computer software and hardware support.

2.4.2 Three famous evaluation programs

✖ Linpack

- + The entire project was developed by J. J. Dongarra hosted and spoke with J. R. Bunch, C. B. Moler, and G. W. Stewart and other people from different organizations worked together, and the result was the official release of the Linpack program package in 1984. Initially it was used to measure the performance of supercomputers, the earliest version was written in Fortran77, and later there were versions of C and Java. In fact, part of Linpack's work was done by the Los Alamos Science Laboratory, and it also included the TAMPR system adapted by Jim Boyle and Ken Dritz.

2.4.2 Three famous evaluation programs

✖ Linpack

+ Several important features :

- ✖ The first to pioneer Mechanics analysis software.
- ✖ Software libraries are provided, and ordinary users can modify them by themselves to meet special needs.
- ✖ The first to establish a comparative standard for mathematical software, especially the measurement method of FLOPS. If a computer system can execute this program correctly, then even if the hardware itself does not have a floating-point circuit at all, its FLOPS value can still be measured.
- ✖ It is suitable for various computer systems, especially since it is an efficient computing program in itself, so it is still widely used to solve various mathematical and engineering problems today, and even common mathematical software such as MATLAB on the market also uses it to deal with matrix operation problems.

2.4.2 Three famous evaluation programs

✖ Dhrystone

- + The first two evaluation programs are based on floating-point operations, while the last one to be introduced, Dhrystone, focuses on string control and integer operations, which was published by Reinford P. Weicher of Siemens Nixdorr Information Company in 1984. The main purpose of this program is to test CPU performance, so it does not include actions such as output/input (I/O) or system calls. The most special thing is that the method of expressing the measurement results is directly in Dhrystone, unlike Whetstone, which is presented in WIPS.

2.4.3 Problems with the evaluation process

- ✖ The above three famous performance evaluation programs are quite simple in terms of purpose and writing, and should have been quite good performance reference standard programs.
- ✖ Unfortunately, because their computing patterns are too clear, many manufacturers of the tested systems write compilers specifically for the characteristics of these programs in order to obtain better test data, such as deliberately "optimizing" the program into a very small machine code object, so that it is almost always stored in cache during execution, thus obtaining results that exceed normal speed.

2.4 Standard procedure for evaluating performance

- ✖ Understand the performance evaluation process
(Benchmark)
- ✖ Three famous evaluation programs
- ✖ Problems with the evaluation process

2.4.3 Problems with the evaluation process

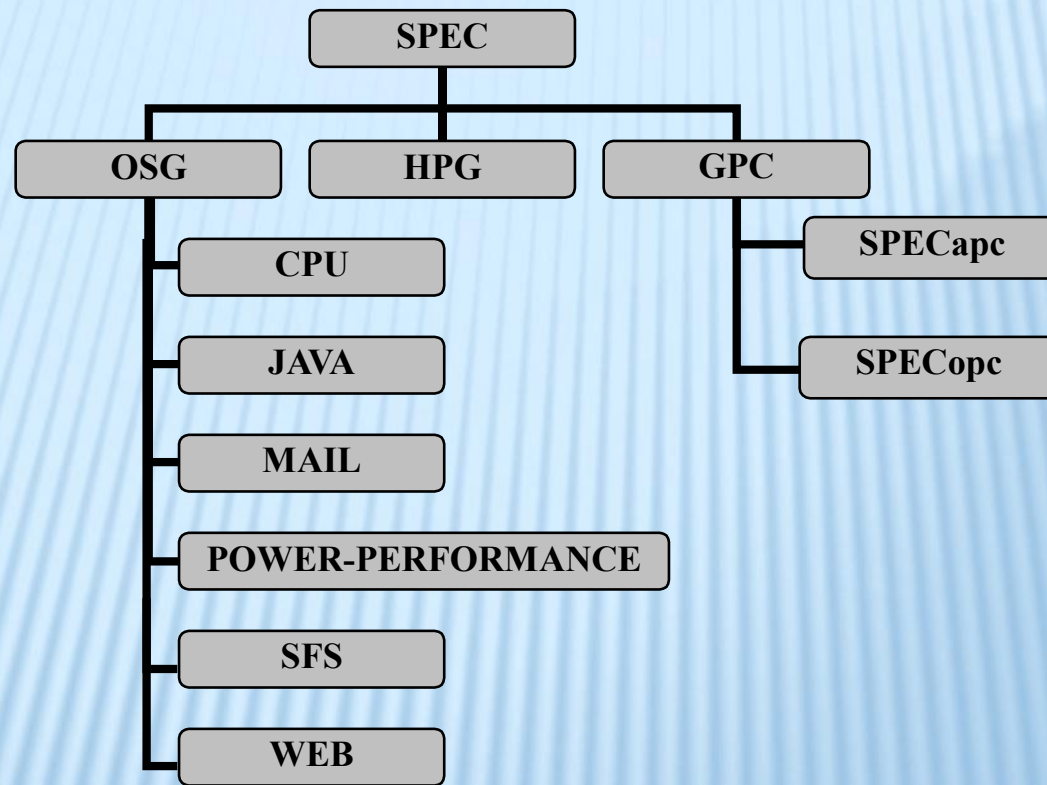
- ✖ This approach is deceptive to testers, as the result data does not provide sufficient and accurate performance information, and the credibility of the evaluation program is questioned to a considerable extent.

2.5 Performance evaluation standard organization

✖ The most famous international standards organization – SPEC

- + 「Standard Performance Evaluation Corporation」 ◦
- + A non-profit organization co-founded by some workstation manufacturers, such as Sun, DEC, MIPS, etc.
- + Create, maintain, and endorse a meaningful standard evaluation process for newly introduced, high-performance computers.

2.5 Performance evaluation standard organization



SPEC's org chart

2.5 Performance evaluation standard organization

- ✖ Open Systems Committee (OSG)
- ✖ High Efficiency Systems Committee (HPG)
- ✖ Drafting and Workstation Effectiveness Committee (GWPG)

2.5.1 Open Systems Committee (OSG)

- ✖ OSG (The Open System Group) It was originally established by SPEC and focused on desktop computer systems, high-end workstations, and servers.
- ✖ Under the OSG, six subcommittees have been established, each providing different evaluation procedures (groups) :
 - + CPU 、 JAVA 、 MAIL 、 POWER-PERFORMANCE 、 SFS
、 WEB

2.5.1 Open Systems Committee (OSG)

- ✖ CPU — Provide CPU performance evaluation programs, such as SPECint, SPECfp, SPECrates, etc ◦
- ✖ JAVA — Provide performance evaluation programs on both the client and server sides of Java, such as JVM98, JBB2005, SPECjAppServer2004.
- ✖ MAIL — Provide an Internet Service Provider (ISP) evaluation program, such as SPECmail2001.

2.5.1 Open Systems Committee (OSG)

- ✖ OOWER-PERFORMANCE — The first generation of SPEC evaluation programs for evaluating the energy efficiency of server-level computers has been developed SPECpower_ssj2008.
- ✖ SFS — Provide a performance evaluation process for file services , 如 SFS93 、 SFS97 、 SFS2008 。
- ✖ Provide a performance evaluation program for web servers, such as WEB96 、 WEB99 、 WEB99_SSL WEB2005 。

2.5 Performance evaluation standard organization

- ✖ Open Systems Committee (OSG)
- ✖ High Efficiency Systems Committee (HPG)
- ✖ Drafting and Workstation Effectiveness Committee (GWPG)

2.5.2 High Efficiency Systems Committee (HPG)

- ✖ HPG (High-Performance Group) The performance evaluation program provided is mainly for high-performance system architectures, such as peer-to-peer multiprocessor systems, workstation clusters, distributed parallel memory systems, and parallel supercomputers.
- ✖ HPG It can be said to be a cross-platform standard evaluation organization, providing performance evaluation programs that use industry-standard application programming interfaces and are all computational work for science and technology.

2.5 Performance evaluation standard organization

- ✖ Open Systems Committee (OSG)
- ✖ High Efficiency Systems Committee (HPG)
- ✖ Drafting and Workstation Effectiveness Committee
(GWPG)

2.5.3 Drafting and Workstation Effectiveness Committee (GWPG)

✖ GWPG (Graphics & Workstation Performance Group)

The following are two separate project organizations responsible for developing graphical and workstation performance evaluation procedures, as well as routines for performance reporting:

- + SPECapc

- + SPECgpc

2.5.3 Drafting and Workstation Effectiveness Committee (GWPG)

- ✖ SPECapc (SPEC Application Performance Characterization)

- + Founded in 1997, it provides performance evaluation programs that emphasize graphics applications, mostly used in CAD/CAM or general visualization applications.

- ✖ SPECgpc (SPEC Graphics Performance Characterization)

- + Founded in 1993, the main focus is on the evaluation of OpenGL performance, providing performance evaluation programs that mainly run OpenGL application programming interfaces (APIs).

Exercises

- ✖ 1 Please explain the following terms: (1) Response time (2) Execution time (3) Throughput (4) Benchmark

Exercises

Sol:

- (1) Response time: How long does it take for the first response to be generated when the required work is submitted.
- (2) Execution time: The time it takes for a job to run from start to finish.
- (3) Generate: The maximum number of times a specific task can be completed per unit of time.
- (4) Evaluation program: A set of hybrid programs that cover various calculations and are used to evaluate system performance is called a set of evaluation programs. The single program that makes up the review program group is called the evaluation program.

Exercises

2. Every three years, the number of transistors in the computer increases by 4 times, and the performance (speed) of the CPU is directly proportional to the internal transistors and the maximum clock frequency. In 1996, the x586/200 CPU contained 3 million transistors with a clock frequency of 200MHz. If an x1486/800 CPU with an 800MHz clock frequency is released in 2002, how many times faster will it be than x586/200?

Exercises

Sol:

$$\frac{800\text{MHz}}{200\text{MHz}} = 4 \text{ times}$$

Exercises

- ✖ 3. Try to explain the relationship between the execution speed of a computer and the amount of generation. What are the similarities and differences between the two?

Exercises

Sol:

(1) The relationship between execution speed and generation volume:

The generation volume can find the average time to complete each job.

(2) The difference between the two:

The average time does not necessarily equal the actual execution time of each job, as the computer itself may have improved the structure of the execution of instructions. For example, using pipeline processing techniques can shorten the total time required to complete multiple tasks. But for a single job, its progress time has not been shortened.

HW#2

1. Suppose we have two processors using the same instruction set, where processor A has a clock period of 1ns and a CPI of 2.5, and processor B has a clock period of 2 ns and a CPI of 1.5 under the same program. How much faster?

HW#2

2. (1) What is Anderson's theorem? According to this theorem, what do the x, y, z parameters represent in the following formula?

$$speedup = \frac{1}{x + (y / z)}$$

(2) If 20% of the system can use parallel processing to improve performance by 50%, what is the overall system growth rate after improving that part with parallel processing?

HW#2

3. The performance of a P microprocessor with 100 MHz was evaluated by executing 10,000,000 evaluation program instructions, and it was found that it took 0.25 seconds. What are the CPI and MIPS data during this performance experiment?